



Google A2A Protocol Cheat Sheet

How AI Agents Talk to Each Other
From Zero to Multi-Agent Systems

Swipe to master A2A →



AI Coach John

FOLLOW



PROITBRIDGE
Strive For Better Future



Repost

THE PROBLEM

AI Agents Were **Isolated Islands**

Before A2A, agents could use tools but could not talk to other agents. Every framework was its own closed world with no standard way to connect.

GAPS THAT EXISTED

- × No standard way for agents to discover each other
- × A LangChain agent could not delegate to a CrewAI agent
- × No common message format across frameworks
- × Duplicate tools had to be built for every agent team
- × Complex tasks required one giant agent instead of specialists

WHAT A2A CHANGES

- ✓ Any agent can discover and call any other agent
- ✓ One shared protocol across all frameworks
- ✓ Specialists collaborate without rebuilding tools

 **A2A is the universal language that finally connects all agents**



AI Coach John

FOLLOW



PROITBRIDGE
Strive For Better Future

 Repost

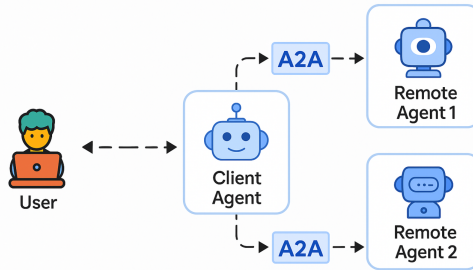
What is Google A2A?

A2A (Agent2Agent) is an open protocol by Google. It gives AI agents a **standard way to communicate over HTTP**, regardless of which framework built them.

💡 Think of HTTP for websites. Any browser loads any website because they all speak HTTP. A2A does the same for agents: any agent can talk to any other agent.

KEY FACTS

- ✓ Released by Google in April 2025
- ✓ Open source under Apache 2.0 license
- ✓ Uses JSON over HTTP, familiar to every developer
- ✓ Works with LangChain, CrewAI, AutoGen, and any framework
- ✓ Supports real-time streaming and long-running tasks
- ✓ 50+ companies contributed at launch including Salesforce and SAP



A2A vs MCP: Know the Difference

These two protocols work together. They are not competitors. Each one solves a different problem.

MCP

Agent talks to Tools

- Web search
- Code execution
- Databases, APIs
- File systems

A2A

Agent talks to Agents

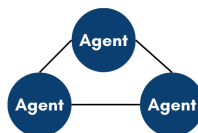
- Delegate subtasks
- Parallel work
- Specialization
- Cross-framework

 MCP gives your agent **hands** to use tools. A2A gives your agent **teammates** to delegate work to. You need both.

A2A VS. MCP EXPLAINED SIMPLY



MODEL CONTEXT PROTOCOL



AGENT-TO-AGENT PROTOCOL

★ Rule: use MCP for tools, use A2A for agent collaboration



AI Coach John

FOLLOW



PROITBRIDGE
Strive For Better Future

 Repost

4 Things You **Must Know** in A2A

Every A2A interaction is built from these four concepts. Learn these and everything else becomes clear.



Agent Card

The agent's public identity and capabilities JSON file



Task

A unit of work sent from one agent to another



Message

Back-and-forth exchange within a running Task



Artifact

The final output or result returned by the agent



Every A2A exchange uses all four. Missing one means a broken integration.



AI Coach John

FOLLOW



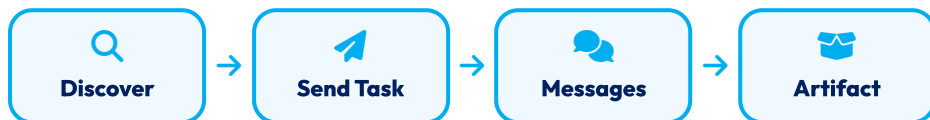
PROITBRIDGE
Strive For Better Future



Repost

The A2A Communication Flow

Every A2A exchange follows the same four-step pattern, from discovery all the way to the final result:



☰ EXAMPLE: "BOOK ME A FLIGHT TO NYC"

- **Discover:** Find a travel agent using its Agent Card
- **Send Task:** Send the booking request as a Task
- **Messages:** Agent asks for travel dates and preferences
- **Artifact:** Confirmed booking details returned as result

i The client agent drives the flow. The server agent handles the work.



AI Coach John

FOLLOW



PROITBRIDGE
Solve For Better Future

🔄 Repost

The Agent Card Explained

Every A2A agent publishes a JSON file at `/well-known/agent.json`. This is how other agents discover it and understand what it can do.

```
{
  "name": "FlightBookingAgent",
  "description": "Books flights, returns confirmations",
  "url": "https://agents.example.com/flights",
  "version": "1.0.0",
  "skills": [
    {
      "id": "book_flight",
      "name": "Book a Flight",
      "description": "Books a flight given origin,
        destination, and travel date"
    }
  ],
  "authentication": {
    "schemes": ["Bearer"]
  }
}
```

i The `skills` description tells other agents **WHEN** to call this agent



AI Coach John

FOLLOW



PROITBRIDGE
Strive For Better Future

Repost

How a Task Moves Through States

A Task is not instant. It moves through states as the agent works on it. Know these to build robust integrations.

-  **submitted** Task received, queued and waiting to start
-  **working** Agent is actively processing the task right now
-  **input-required** Agent paused, needs more info from the caller
-  **completed** Task finished successfully, Artifact is ready
-  **failed / cancelled** Task ended with an error or was stopped early



Always handle **input-required** and **failed** states or your agent will break in production



AI Coach John

FOLLOW



PROITBRIDGE
Strive For Better Future



Repost

Your First A2A Agent in Python

Google provides an official Python SDK. Build a working A2A server agent in under 20 lines.

```
pip install a2a-sdk

from a2a.server import A2AServer, TaskHandler
from a2a.types import Task, Artifact

class WeatherAgent(TaskHandler):
    async def handle_task(self, task: Task):

        # Read the user input from the task history
        user_input = task.history[0].parts[0].text

        # Do your work here (call an API, run a tool...)
        result = f"It is sunny and 25C in {user_input}"

        # Return an Artifact containing the answer
        return Artifact(parts=[{"text": result}])

server = A2AServer(
    agent_card_path="agent_card.json",
    handler=WeatherAgent()
)
server.run(port=8080)
```



Any HTTP client can now call your agent using the standard A2A protocol



AI Coach John

FOLLOW



PROITBRIDGE
Strive For Better Future



Repost

Where A2A Shines in the Real World

A2A is powerful when one agent needs to delegate specialized work to other agent teams. Here are four patterns you will see everywhere:



Travel Booking

Orchestrator delegates to a Flight Agent and Hotel Agent running in parallel



Research Pipeline

Manager Agent sends work to a Search Agent, then a Summarizer Agent



Customer Support

Triage Agent routes each ticket to the right specialist agent per topic



Software Development

Planning Agent delegates to a Coder Agent and a Testing Agent



Pattern: one orchestrator, multiple specialists, each with one clear job



AI Coach John

FOLLOW



PROITBRIDGE
Strive For Better Future



Repost

When Should You Use A2A?

A2A adds power but also adds complexity. Use this guide to decide when it is worth it:

✓ USE A2A WHEN...

- ✓ You need specialist agents built by different teams
- ✓ Tasks can run in parallel across multiple agents
- ✓ You need to mix agents from different frameworks
- ✓ One agent needs to delegate and wait for results

⚠ SKIP A2A WHEN...

- ✗ One agent with MCP tools is enough for the job
- ✗ All agents live in the same framework and codebase
- ✗ Latency is critical and extra network hops matter



Rule: if your agent is getting too big, split it with A2A. If it is simple enough, keep it simple.



AI Coach John

FOLLOW



PROITBRIDGE
Strive For Better Future



Repost

A2A Vocab Cheat Sheet

Bookmark this. Every term you will encounter when working with A2A, explained in one sentence.

A2A

Open protocol by Google for agent-to-agent communication

Agent Card

JSON file that declares what an agent can do

Task

A unit of work with an ID, history, and state

Message

One turn of conversation within a Task

Artifact

The final output that a server agent returns

Client Agent

The agent that sends tasks and drives the work

Server Agent

The agent that receives tasks and produces results

Orchestrator

A manager agent that coordinates multiple sub-agents

Skill

A named capability declared inside the Agent Card



AI Coach John

FOLLOW



PROITBRIDGE
Strive For Better Future

 Repost

Your A2A Learning Roadmap

Everything in this carousel, in order. Follow this path and you will be building multi-agent systems today.

- | | |
|---|------------------------------|
| 1. Understand why A2A was needed | <- isolation problem |
| 2. Learn what A2A is | <- protocol over HTTP |
| 3. Know A2A vs MCP | <- agents vs tools |
| 4. Master the 4 building blocks
card/task/message/artifact | <- |
| 5. Follow the communication flow | <- discover to artifact |
| 6. Read an Agent Card | <- JSON at /.well-known/ |
| 7. Understand Task states | <- submitted to completed |
| 8. Build your first A2A agent | <- Python SDK |
| 9. Explore real use cases | <- travel, research, support |
| 10. Decide when A2A fits | <- avoid over-engineering |
| 11. Learn the vocabulary | <- talk the talk |

 **Save it. Share it. Start Building Today.**

Follow **AI Coach John** for more AI content



AI Coach John

FOLLOW



PROITBRIDGE
Strive For Better Future

 **Repost**